Lab Report

CI/CD with Render: Managed PostgreSQL, Staging Environment & Automated Deployment

DreamCatcher — AI Dream Journal with Continuous Delivery Pipeline

Achindra Sharma — 2547105 (Individual Submission)

4MCA A — MCA Trimester 5: DevOps Lab (Lab 5)

Repository: github.com/Achindra2003/dream-catcher

Production: https://dream-catcher-2-qm7w.onrender.com

Staging: https://dream-catcher-stage.onrender.com

CI/CD Status: **GitHub Actions Run #41588306372 (main) — Passed**

1. Project Description & System Context

DreamCatcher is a Node.js/Express web application that lets a user record a dream and receive an AI-generated interpretation. The goal of this lab is to take that application through a complete CI/CD workflow on **Render**: deploy it to a public domain, ship a change through Git, replace its ephemeral SQLite storage with a managed **PostgreSQL** service, develop and test the change locally, and finally promote it through a separate **staging** environment into production by merging a `stage` branch.

Initially the app stored dreams in a local SQLite file. Render's free web services have an ephemeral filesystem, so every restart or redeploy would erase the journal. Moving to a managed database was therefore the central engineering change of this lab.

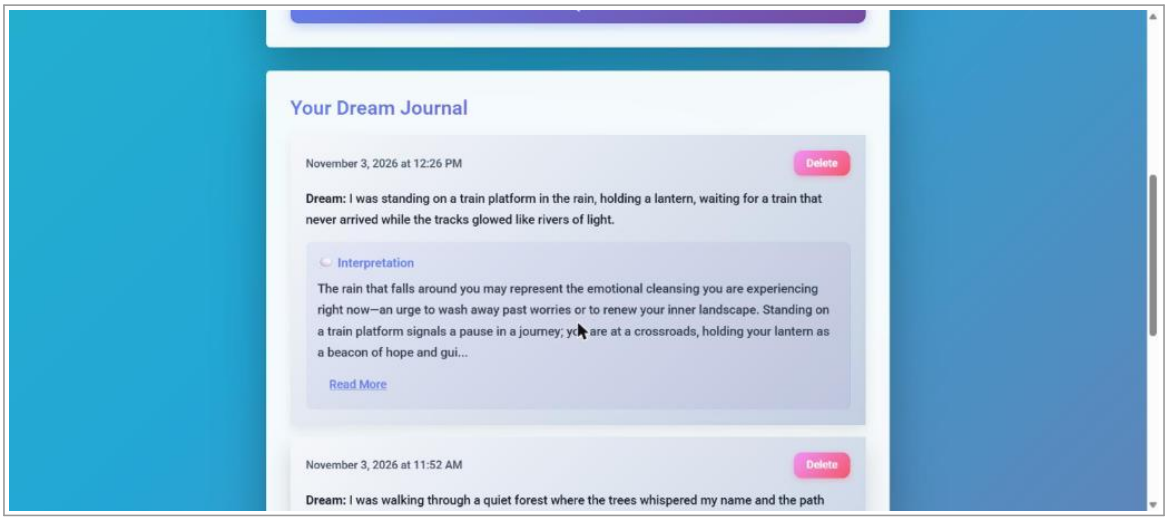


Figure 1 — Step 1 — The production domain (dream-catcher-2-qm7w.onrender.com) with a new dream added through the browser and its AI interpretation displayed.

2. Git Workflow & Continuous Deployment (Step 2)

A visible change was made to the project source (footer branding changed from “DreamSnatcher” to “DreamCatcher by Achindra Sharma (2547105)”), committed, and pushed to `main`. Because auto-deploy is enabled on the Render service, the push triggered a new build with no manual action (trigger = *new commit*).

```
$ git diff --stat
 public/index.html | 2 +-

$ git commit -m "Update footer branding to DreamCatcher with author credit"
$ git push origin main

$ render deploys list      # row for the pushed commit
Commit 4fb2079   Trigger: new commit   Created 05:48:50Z   Finished 05:49:08Z   -> Live in 18 s
```

3. Managed PostgreSQL on Render (Step 3)

A Render PostgreSQL instance **dream-catcher-db** (Free plan, Oregon, Postgres 18) was created. The web service reaches it over Render's private network through the *Internal Database URL*, while local development uses the *External Database URL* (TLS). The connection string is injected as the `DATABASE_URL` environment variable and is never committed to Git (`.env` is git-ignored). A second, independent database, `dreams_stage`, was created on the same instance to isolate staging data from production.

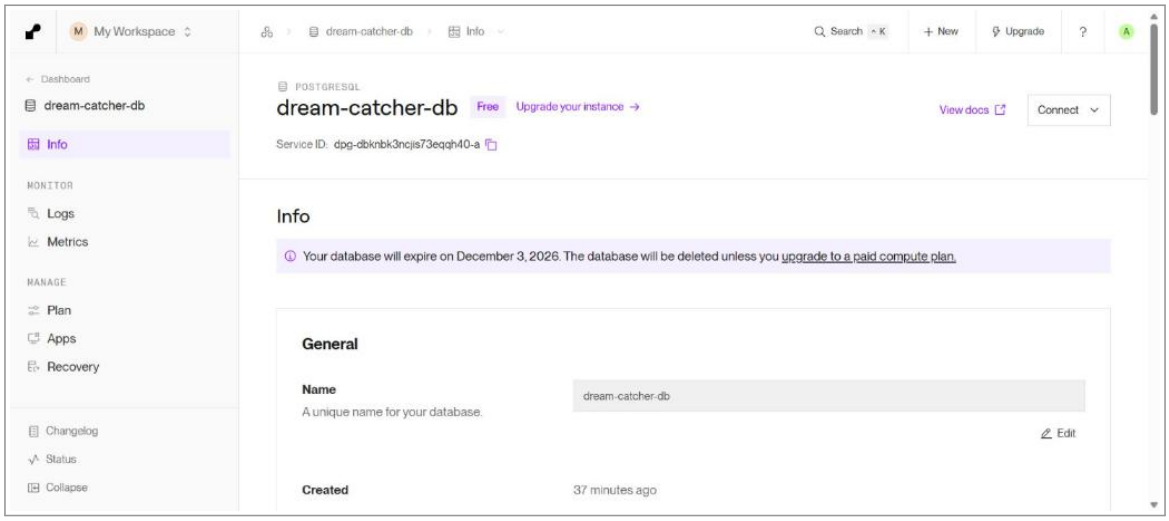


Figure 2 — Render dashboard — the managed PostgreSQL service dream-catcher-db (Free plan).

```
          List of relations
 Schema |  Name  | Type  |    Owner
--------+--------+-------+-------------
 public | dreams | table | dreams_user
(1 row)

                                      Table "public.dreams"
     Column     |           Type           | Collation | Nullable |              Default
----------------+--------------------------+-----------+----------+-----------------------------------
 id             | integer                  |           | not null | nextval('dreams_id_seq'::regclass)
 dream_text     | text                     |           | not null |
 interpretation | text                     |           |          |
 created_at     | timestamp with time zone |           | not null | now()
Indexes:
    "dreams_pkey" PRIMARY KEY, btree (id)

 id |                    dream                     |          created_at
----+----------------------------------------------+-------------------------------
  1 | Local test: a library where every book is a d | 2026-11-03 05:52:10.526668+00
  2 | I was walking through a quiet forest where th | 2026-11-03 06:22:58.930375+00
  3 | I was standing on a train platform in the rai | 2026-11-03 06:56:43.243198+00
(3 rows)
```

4. Code Changes & Local Development (Step 4)

The persistence layer was migrated from SQLite to PostgreSQL and the application was run locally against the Render database. The key changes are summarised below.

Area	Before	After
Driver	sqlite + sqlite3 (native build)	pg (pure JavaScript, connection pool)
Connection	Local file dreams.db	DATABASE_URL, TLS enabled for remote hosts
Schema	INTEGER PRIMARY KEY AUTOINCREMENT, DATETIME	SERIAL PRIMARY KEY, TIMESTAMPTZ DEFAULT now()
Queries	? placeholders, db.run / db.get	$1 placeholders, pool.query ... RETURNING *
App structure	server.js did everything	app.js (Express app) + server.js (boot) so it can be tested
Probes	none	GET /health (liveness) and GET /ready (database check)

Five automated tests (Node's built-in `node:test`) cover input validation and the health endpoint. The app was then started locally and connected to the Render Postgres instance:

```
ok 1 - GET /health returns ok
ok 2 - rejects non-string input
ok 3 - rejects empty / whitespace-only text
ok 4 - rejects text over 5000 characters
ok 5 - trims and accepts valid text
# tests 5
# pass 5
# fail 0

$ npm start
Database configured: true
Server running on http://localhost:3001

$ curl localhost:3001/health   -> {"status":"ok","uptime":4}
$ curl localhost:3001/ready    -> {"status":"ready","database":"up"}
```

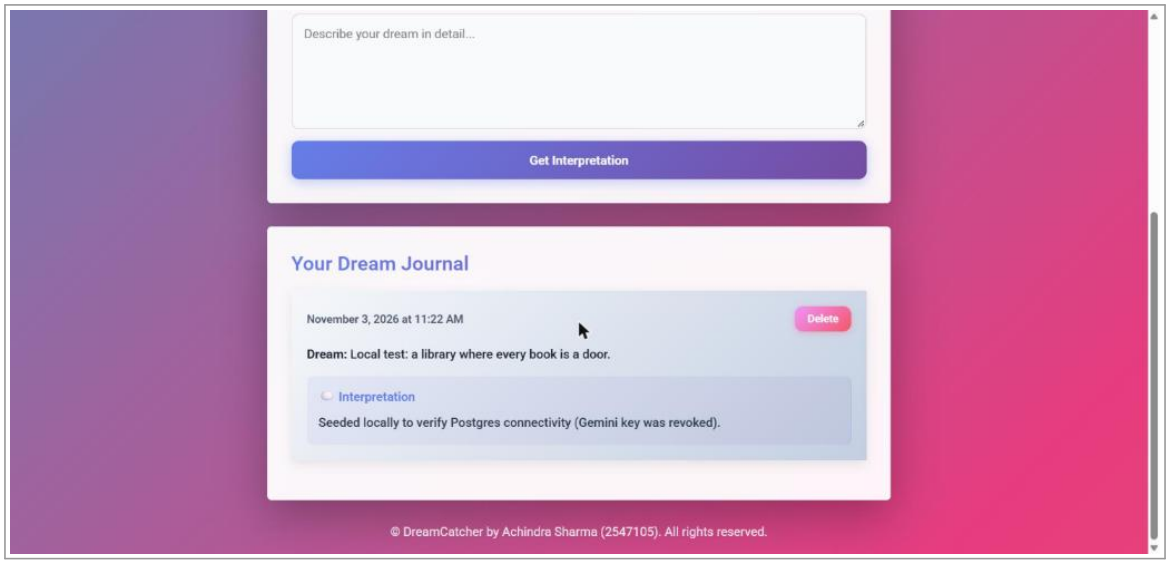


Figure 3 — Local run (localhost:3001) reading a row stored in the Render PostgreSQL database.

Note: the row shown above was inserted directly with `psql` because the Gemini API key in the local environment had been revoked by Google at that time (HTTP 403, “key was reported as leaked”). The AI provider was subsequently replaced (see Section 7), after which interpretations were generated end to end on staging and production.

5. Staging Environment: the `stage` Branch (Step 5)

A `stage` branch was created and pushed. A second Render web service, **dream-catcher-stage**, was created from the same repository but pinned to the `stage` branch (Docker runtime, Free plan). It has its own environment variables and its own database (`dreams_stage`), so testing cannot corrupt production data.

	Production	**Staging**
Render service	dream-catcher-2	dream-catcher-stage
Git branch	main	stage
Runtime	Docker (multi-stage Dockerfile)	Docker (multi-stage Dockerfile)
Database	dream-catcher-db / dreams	dream-catcher-db / dreams_stage
URL	dream-catcher-2-qm7w.onrender.com	dream-catcher-stage.onrender.com

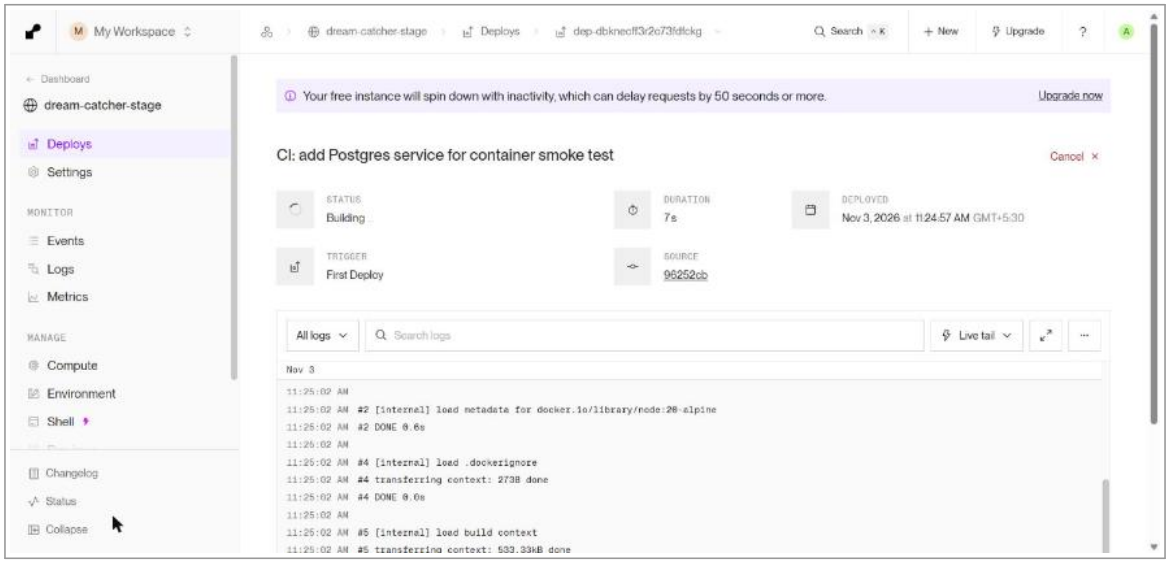


Figure 4 — Render building dream-catcher-stage from the stage branch (first deploy).

Once the Groq API key and stage database URL were configured, the service went live. The stage endpoints returned healthy responses and a new dream was interpreted and stored in the stage database:

```
$ curl https://dream-catcher-stage.onrender.com/health
{"status":"ok","uptime":107}

$ curl https://dream-catcher-stage.onrender.com/ready
{"status":"ready","database":"up"}

$ curl -X POST .../api/dreams -d '{"dream_text":"I was flying over a glowing ocean..."}'
{"id":1,"dream_text":"I was flying over a glowing ocean...","interpretation":"Flying in a dream often signal
```

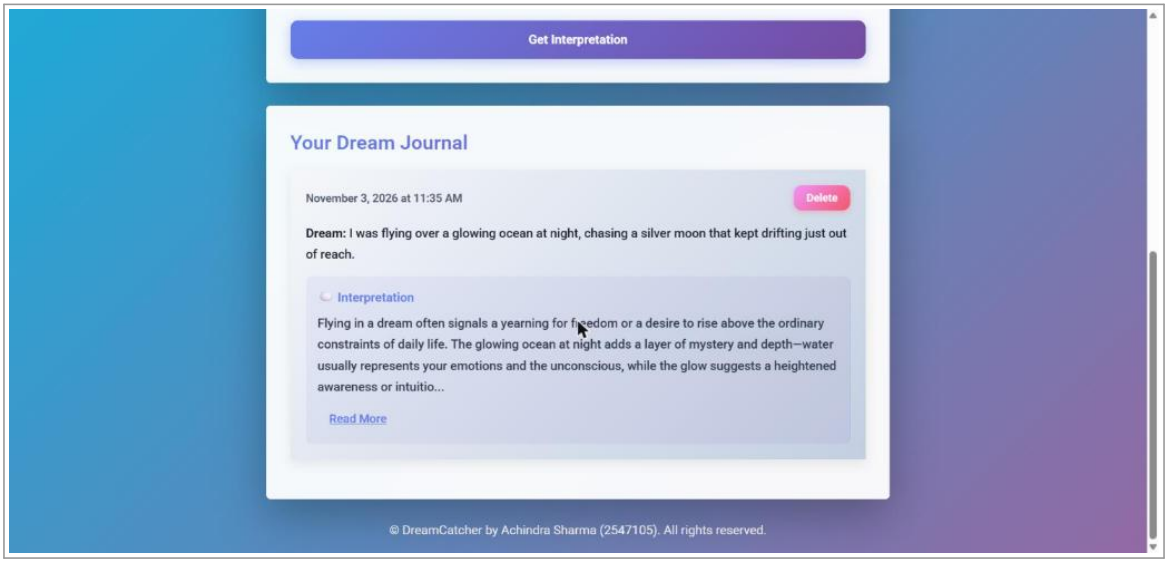


Figure 5 — The staging site working end to end (Groq interpretation + Postgres persistence).

6. Merge to Production & Persistence Proof

After verification on staging, a pull request from `stage` to `main` was opened. GitHub Actions ran automatically on the PR; once green, the PR was merged, which triggered the production auto-deploy.

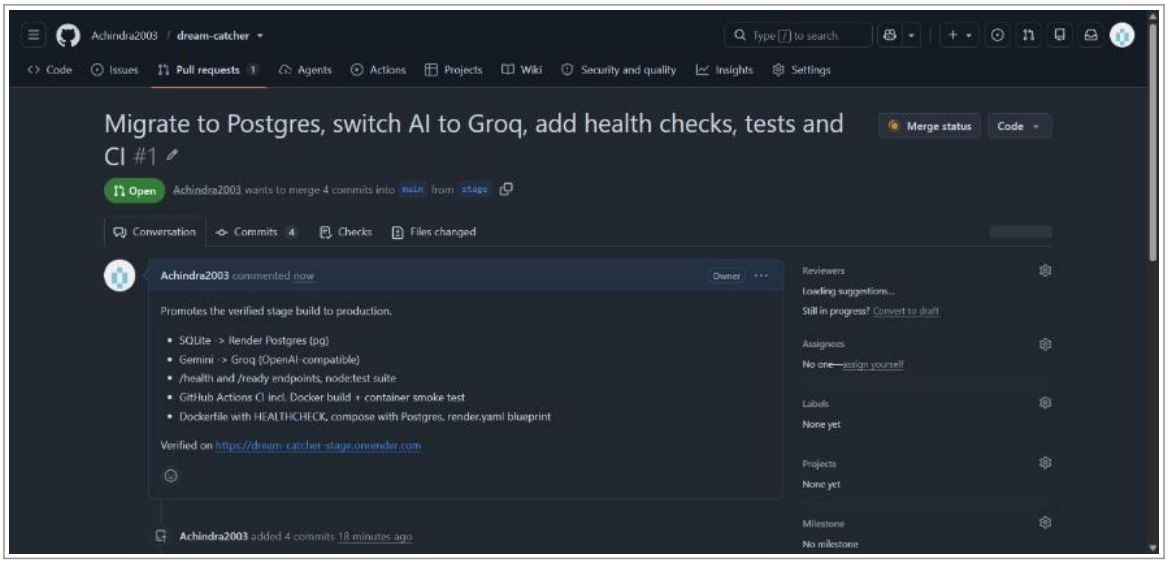


Figure 6 — Pull request #1 (stage -> main) opened with a summary of the changes.

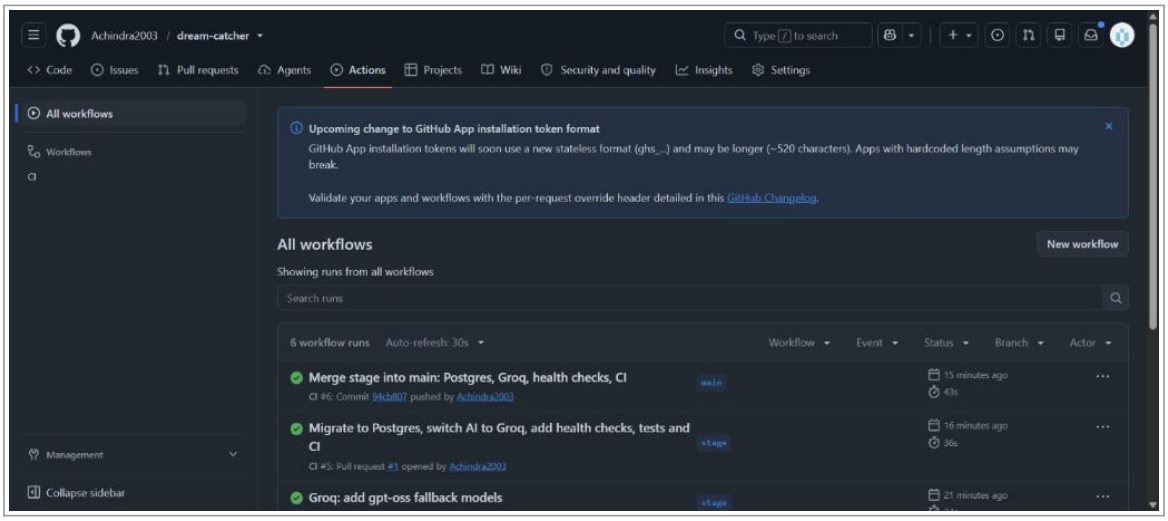


Figure 7 — GitHub Actions run history: all runs after the workflow fix passed, including CI #6 on main after the merge.

```
success  main  push  Merge stage into main: Postgres, Groq, health checks, CI
success  stage  pull_request  Migrate to Postgres, switch AI to Groq, add health checks, tests and CI
success  stage  push  Groq: add gpt-oss fallback models
success  stage  push  Switch AI provider from Gemini to Groq (OpenAI-compatible, Llama 3.3)
success  stage  push  CI: add Postgres service for container smoke test
failure  stage  push  Migrate to Render Postgres, add /health, tests, CI workflow and compo…

*   94cb807 Merge stage into main: Postgres, Groq, health checks, CI
|\
| * a4d6a17 Groq: add gpt-oss fallback models
| * cd3bdb4 Switch AI provider from Gemini to Groq (OpenAI-compatible, Llama 3.3)
| * 96252cb CI: add Postgres service for container smoke test
| * 9d213fc Migrate to Render Postgres, add /health, tests, CI workflow and compose Postgres
|/
* 4fb2079 Update footer branding to DreamCatcher with author credit
* debcef9 Dockerize dream-catcher with production multi-stage Dockerfile and Render Docker runtime
* e1f7654 Add gemini-3.5-flash and multi-model fallback to resolve 404/503 model availability
* 30b898e Update default Gemini model to gemini-3.8-flash
* 85c87c5 Configure production build command and dev scripts for Render deployment
* 9429364 Initial commit: Dream Catcher app with Gemini AI interpretation
```

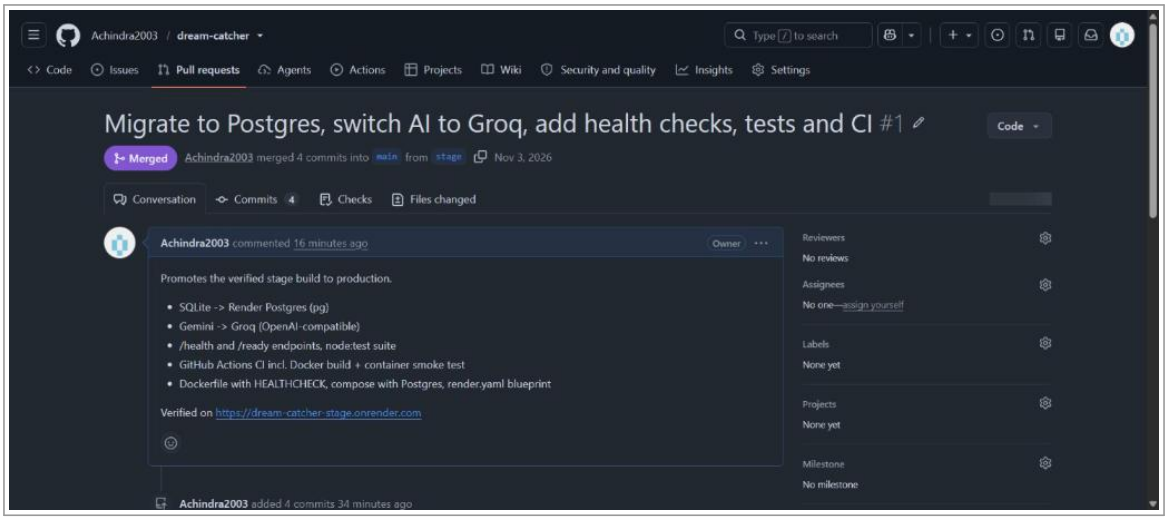


Figure 8 — Pull request #1 merged into main (merge commit 94cb807).

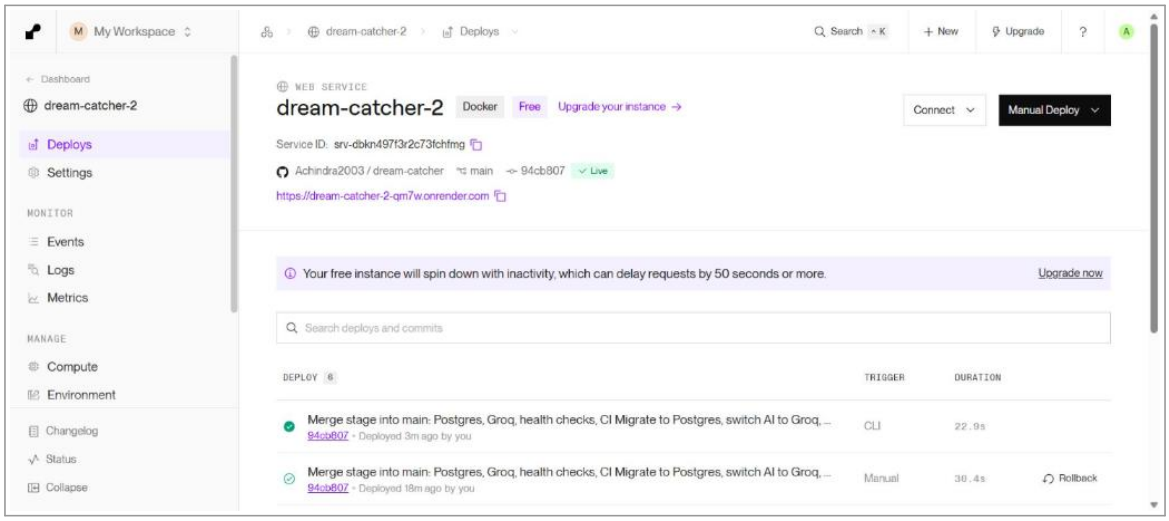


Figure 9 — Render production deploy history: commit 94cb807 deployed and Live.

Persistence test. The decisive reason for adopting PostgreSQL is that data must survive a redeploy. Two dreams existed in the production database; a full redeploy of the service was triggered through the Render CLI, and the API was queried again afterwards:

```
BEFORE redeploy:  [(2, 'I was walking through a quiet fores'), (1, 'Local test: a library where every b')]

$ render deploys create srv-dbkn497f3r2c73fchfmg
Created deploy dep-dbknrie7bikc73fdt8ug   (trigger: api)   Live 06:23:28Z

AFTER redeploy:   [(2, 'I was walking through a quiet fores'), (1, 'Local test: a library where every b')]
```

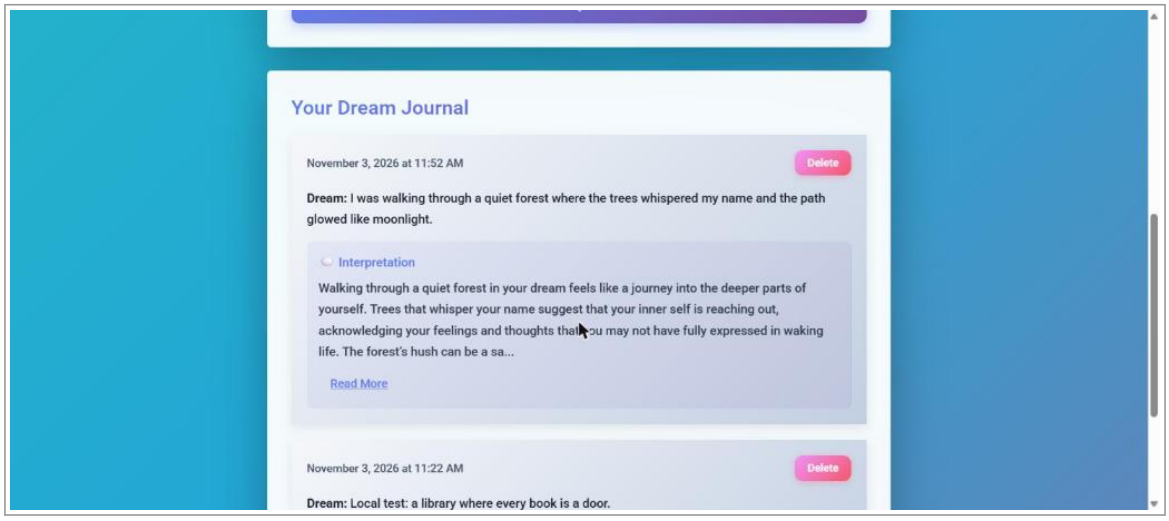


Figure 10 — Production site after the redeploy — both dreams are still present (they would have been lost with SQLite).

7. Beyond the Baseline: Self-Learning Initiatives

In addition to the five required steps, the following initiatives were implemented:

1. **Automated CI Pipeline (GitHub Actions)** — `.github/workflows/ci.yml` runs on every push and pull request: install, run tests, build the Docker image, then start the container next to a Postgres service container and call `/health` and `/ready` as a smoke test.
2. **Automated Tests** — five `node:test` cases (validation rules and the health endpoint) gate every change; the app was refactored into `app.js` + `server.js` to make it testable.

3. **Liveness & Readiness Probes** — `/health` (process is up) and `/ready` (database reachable), plus a Docker `HEALTHCHECK` instruction.
4. **Hardened Multi-Stage Docker Image** — two-stage build on `node:20-alpine`, non-root `USER node`, OCI labels. Removing the native SQLite compile step also removed the need for a compiler toolchain in the build.
5. **Environment Separation** — distinct staging and production services and databases; secrets (API key, database URL) live only in Render environment variables, never in Git.
6. **Infrastructure as Code** — `render.yaml` describes both web services and the database as a reference Blueprint, and `docker-compose.yml` now provisions a local Postgres for parity with production.
7. **AI Provider Migration & Resilience** — replaced Gemini with **Groq** through its OpenAI-compatible API and added an ordered model-fallback list, so a single unavailable model no longer breaks the feature.

8. Challenges Encountered & How They Were Resolved

Problem	Root cause	Resolution
Dreams lost on restart	SQLite file on Render's ephemeral disk	Migrated to managed PostgreSQL; verified persistence across a redeploy.
CI failed on first run	Container smoke test had no database, so the app exited at start-up	Added a Postgres service container to the workflow; the next run passed.
AI calls returned 403	Gemini API key flagged as leaked and revoked	Rotated to a new provider (Groq) and kept keys only in environment variables.
Groq returned 404 for Llama models	The account only had access to the gpt-oss models	Read the Render logs, added gpt-oss models to the fallback list, redeployed; interpretations worked.
Stage service would not start	DATABASE_URL and API key were not yet configured	Added the variables; service redeployed and went Live.
Production deploy stuck 17 min	A manual deploy sat in “update in progress” and blocked the queue	It timed out; the queued merge deploy then built and went Live.
Several similarly named services	dream-catcher, -1 and -2 exist in the workspace	Confirmed the active service from deploy history and used dream-catcher-2.

Operational note: Render's free PostgreSQL instances expire 30 days after creation (this one on 3 December 2026) and free web services spin down after inactivity (first request after idle takes about 25–50 seconds).

9. Key Learnings & Engineering Reflections

1. **State Must Live Outside the Container:** An ephemeral filesystem makes file-based databases unsuitable for cloud deployment. A managed database decouples data from the deployment lifecycle, which is what makes frequent redeploys safe.
2. **Staging Reduces Risk:** Promoting a verified build from a separate stage service and database caught configuration problems (missing variables, model access) before they could reach

production users.

3. **Automation Gives Fast Feedback:** Auto-deploy on commit plus a CI workflow turns every push into a tested, reproducible build; the failed first CI run was diagnosed and fixed within minutes from its logs.
4. **Secrets Belong in the Platform, Not the Repo:** The revoked API key showed why credentials must be injected through environment variables and rotated easily without touching the code or Git history.